

Java Backend Architecture

Interview Series

Chapter 9.1

CPU Caches & Memory Hierarchy

50+ Interview Q&A

Code Examples

Architecture Diagrams

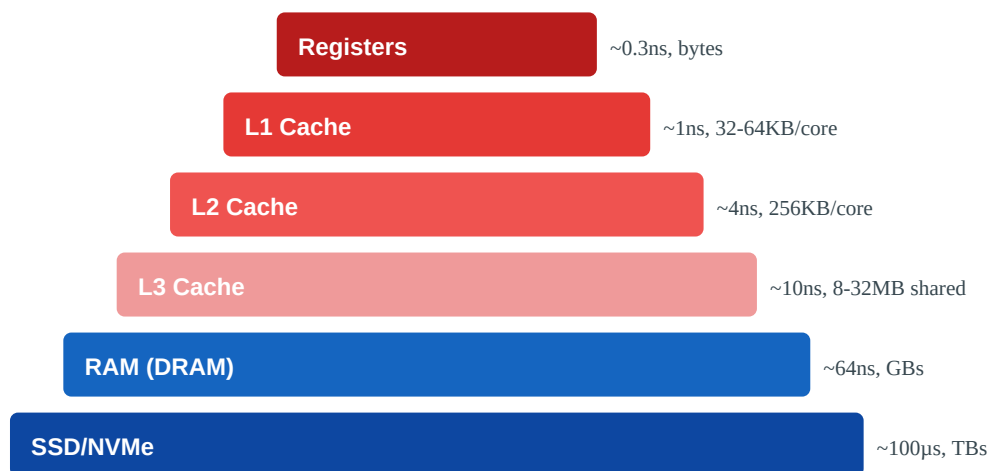
Advanced Topics

Chapter 9.1 — CPU Caches & Memory Hierarchy

Introduction

The memory hierarchy is the foundation of computer performance. Understanding CPU cache levels, cache line mechanics, false sharing, and NUMA architecture is critical for writing high-performance Java code — especially for latency-sensitive services, concurrent data structures, and high-throughput message processing. Cache-aware programming can yield 10-100x performance improvements over cache-oblivious code.

Memory Hierarchy



Rule: 10x latency per level down. L1 miss cost = ~10ns (10 wasted CPU cycles at 1GHz).

Level	Latency	Size	Scope	Managed By
Registers	~0.3 ns	Bytes	Per core	Compiler/CPU
L1 I/D Cache	~1 ns	32–64 KB	Per core	CPU hardware
L2 Cache	~4 ns	256 KB–1 MB	Per core	CPU hardware
L3 Cache	~10–30 ns	8–64 MB	Shared/chip	CPU hardware
Main RAM	~64 ns	GBs	System-wide	OS + CPU MMU
NVMe SSD	~100 µs	TBs	System-wide	OS + filesystem
HDD	~10 ms	TBs	System-wide	OS + filesystem

Cache Line & Spatial Locality

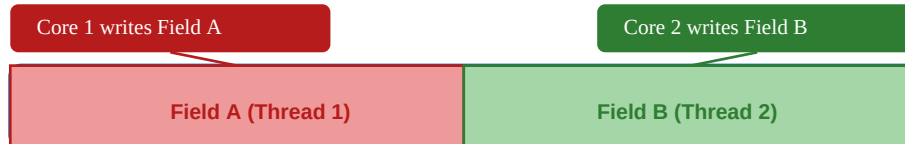
The CPU always loads data in units of **cache lines (64 bytes)**. When you access one field, the entire 64-byte block is loaded into L1. **Spatial locality**: accessing nearby memory (e.g., iterating an array sequentially) maximises cache line utilisation. **Temporal locality**: reusing recently-accessed data before it's evicted. Linked-list traversal has poor cache behaviour — each node can be on a different cache line.

```
// BAD: column-major traversal – cache UNFRIENDLY (jumps by row stride)
int sum = 0;
for (int col = 0; col < N; col++)
    for (int row = 0; row < N; row++)
        sum += matrix[row][col]; // jumps N*4 bytes each iteration

// GOOD: row-major traversal – cache FRIENDLY (sequential access)
int sum = 0;
for (int row = 0; row < N; row++)
    for (int col = 0; col < N; col++)
        sum += matrix[row][col];
```

```
matrix[row][col]; // sequential 4-byte steps // For N=1000: row-major is ~10x faster due to L1/L2 cache hits
```

False Sharing



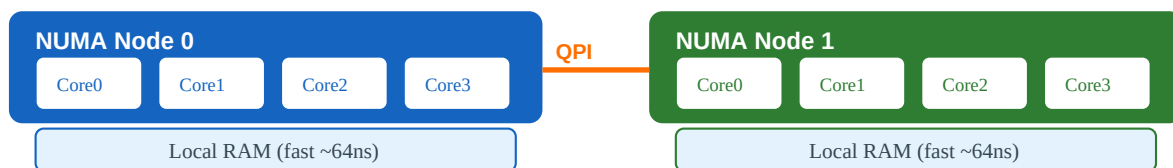
Problem: Both cores share the SAME 64-byte cache line!

Core 1 write invalidates Core 2's cache line → Core 2 must reload from L3/RAM → performance collapse

Fix: pad fields to 64 bytes (@Contended annotation) so A and B land on different cache lines

```
// BAD: false sharing – counter0 and counter1 on same cache line
class BadCounters {
    long counter0 = 0; // bytes 0-7
    long counter1 = 0; // bytes 8-15 // Both fit in one 64-byte cache line!
}
// GOOD: @Contended pads to separate cache lines (JVM flag required)
class GoodCounters {
    @jdk.internal.vm.annotation.Contended long counter0 = 0;
    @jdk.internal.vm.annotation.Contended long counter1 = 0;
}
// Requires: -XX:-RestrictContended JVM flag
// Manual padding alternative:
class PaddedCounter {
    long p1, p2, p3, p4, p5, p6, p7; // 56 bytes of padding
    long counter = 0; // 8 bytes – own cache line
    long p8, p9, p10, p11, p12, p13, p14; // trailing padding
}
```

NUMA (Non-Uniform Memory Access)



NUMA: accessing remote node RAM adds ~100ns latency (2x local). Thread affinity: bind threads to their NUMA node.

Java: -XX:+UseNUMA flag enables NUMA-aware heap allocation (Eden per NUMA node).

Modern multi-socket servers have NUMA topology: each CPU socket has its own memory bank. Accessing local memory (~64ns) is 2x faster than remote (cross-socket) memory (~100-120ns). JVM with **-XX:+UseNUMA** allocates Eden space per NUMA node, reducing cross-socket memory traffic for short-lived objects.

```
# Check NUMA topology on Linux
numactl --hardware
numactl --show
# Bind a Java process to NUMA node 0
numactl --cpunodebind=0 --membind=0 java -jar app.jar
# JVM NUMA-aware GC
java -XX:+UseNUMA -XX:+UseParallelGC -jar app.jar
# Check CPU topology in Java
Runtime.getRuntime().availableProcessors() – logical CPUs
// Use JNA or /proc/cpuinfo for physical NUMA topology
# Linux perf: measure cache misses
perf stat -e cache-misses,cache-references,instructions java -jar app.jar
```

Branch Prediction & Speculative Execution

Modern CPUs speculatively execute instructions ahead of a branch. Mispredicted branches cause a pipeline flush (~15 cycles penalty). **Predictable branches** (always true/false, or alternating in a known pattern) are nearly free. Unpredictable branches (50% probability) are expensive. Sorting data before processing can make branches more predictable.

```
// Branch prediction test: sorted vs unsorted array
int[] data = new int[32768];
Random rng = new Random();
for (int i = 0; i < data.length; i++) data[i] = rng.nextInt(256);
// SLOW: ~6.5s
```

```

- 50% branch misprediction rate long sum = 0; for (int i = 0; i < 100000; i++) for (int c : data) if (c >= 128) sum += c; // FAST: ~1.7s - sort makes branch always-true then always-false
Arrays.sort(data); sum = 0; for (int i = 0; i < 100000; i++) for (int c : data) if (c >= 128) sum += c;

```

JVM Memory Model & CPU Caches

JMM Concept	CPU Cache Mechanism	Java Keyword/API
Visibility	Cache coherence protocol (MESI)	volatile, synchronized
Happens-before	Memory barrier / fence instruction	volatile write/read, lock release/acquire
Atomicity	Cache line lock, CAS instruction	synchronized, AtomicInteger
Reordering prevention	Store/load barriers	volatile, VarHandle with acquire/release
False sharing	Cache line contention	@Contended, manual padding

50+ Interview Questions & Answers

Q1. What is the memory hierarchy and why does it exist?

A hierarchy of storage: registers → L1 cache → L2 cache → L3 cache → RAM → SSD → HDD. Each level trades speed for capacity and cost. Registers are fastest (sub-ns) but tiny; RAM is slower (64ns) but large. The hierarchy exploits temporal and spatial locality.

Q2. What is a CPU cache?

High-speed SRAM memory built into the CPU die. Stores recently-accessed data/instructions to avoid slow RAM fetches. L1: fastest (~1ns, 32-64KB per core). L2: slightly larger (~4ns, 256KB). L3: shared across cores (~10-30ns, MBs).

Q3. What is a cache line?

The fundamental unit of data transfer between RAM and CPU cache — always 64 bytes on x86/ARM. When you access any byte, the entire 64-byte cache line is loaded into the cache.

Q4. What is spatial locality?

Accessing memory locations near recently-accessed locations. Arrays exhibit excellent spatial locality (sequential elements loaded in one cache line). Linked lists exhibit poor spatial locality (nodes scattered in heap).

Q5. What is temporal locality?

Reusing recently-accessed data before it's evicted from cache. Loop variables have high temporal locality. One-time sequential scans have low temporal locality.

Q6. What is false sharing?

Two threads write to different variables that happen to share the same 64-byte cache line. When Thread 1 writes, it invalidates Thread 2's cached copy, forcing a reload from L3/RAM — even though they accessed different data.

Q7. How do you detect false sharing?

Linux perf: 'perf c2c report' or 'perf stat -e cache-misses'. Java: JMH with -prof perfasm or Linux perf profiler. Symptoms: unexpectedly high L3 cache miss rate on writes, poor parallel scaling on multi-core.

Q8. How do you fix false sharing in Java?

Use `@jdk.internal.vm.annotation.Contended` (requires `-XX:-RestrictContended`). Or manually pad: add 7 long fields (56 bytes) before and after the hot field to ensure it occupies its own cache line.

Q9. What is `@Contended` in Java?

An annotation that pads the annotated field or class to occupy its own cache line (64 bytes padding added). Used in `java.util.concurrent` (`LongAdder`, `ConcurrentHashMap` counters). Requires `-XX:-RestrictContended` JVM flag.

Q10. What is NUMA?

Non-Uniform Memory Access — multi-socket systems where each CPU socket has local memory. Accessing local memory is fast (~64ns); cross-socket (remote) memory is slower (~100-120ns). Programs should prefer local memory allocation.

Q11. How does the JVM support NUMA?

`-XX:+UseNUMA` enables NUMA-aware heap allocation: the GC allocates Eden spaces per NUMA node. New objects are allocated on the node local to the allocating thread, reducing cross-socket traffic.

Q12. What is the MESI cache coherence protocol?

Modified/Exclusive/Shared/Invalid — states for cache lines across cores. When one core writes a cache line, all other cores' copies are Invalidated (I state). They must reload from L3 or RAM before the next access.

Q13. What is a memory barrier (fence)?

A CPU instruction that prevents reordering of memory operations across the barrier. Types: store barrier (all stores visible before barrier), load barrier (no loads from before barrier seen after), full barrier (both). Java volatile uses full barriers.

Q14. What is branch prediction and why does it matter for performance?

CPU speculatively executes instructions ahead of a conditional branch. If the prediction is wrong, the pipeline is flushed (~15 cycle penalty). Predictable branches (always true, or patterned) are near-free. 50% unpredictable branches are expensive.

Q15. What is speculative execution?

CPU executes instructions ahead before knowing if they'll be needed (e.g., before a branch resolves). Improves throughput by hiding latency. Spectre/Meltdown are security vulnerabilities exploiting speculative execution to read privileged memory.

Q16. What is instruction-level parallelism (ILP)?

CPU executes multiple independent instructions simultaneously using multiple execution units. Out-of-order execution reorders independent instructions to maximise pipeline utilisation.

Q17. What is a TLB (Translation Lookaside Buffer)?

A cache for virtual-to-physical address translations. TLB miss = page table walk (~100ns). Large pages (hugepages: 2MB/1GB) reduce TLB pressure by covering more memory per entry. Java: `-XX:+UseHugeTLBFS` or `-XX:+UseLargePages`.

Q18. What are hugepages and when do you use them in Java?

OS memory pages of 2MB or 1GB (vs 4KB default). Reduce TLB misses for large heap JVM processes. Enable with `-XX:+UseLargePages` `-XX:LargePageSizeInBytes=2m`. Most beneficial for heap > 4GB with many long-lived objects.

Q19. What is prefetching?

CPU predicts future memory accesses and loads data into cache before it's needed. Software prefetch: `__builtin_prefetch()` in C. Java: the JIT compiler inserts prefetch instructions for array iteration. Hardware prefetcher handles sequential patterns automatically.

Q20. What is the difference between L1 instruction cache and L1 data cache?

Modern CPUs have separate L1 caches: L1-I (instruction cache) and L1-D (data cache), each typically 32KB. L2 and L3 are usually unified. Hot code paths benefit from instruction cache — avoid megamorphic virtual dispatch.

Q21. What is cache associativity?

Determines where a cache line can be stored. Direct-mapped: exactly one slot (fast but many conflicts). N-way set-associative: N possible slots (better utilisation). L1 is typically 8-way; L3 is 16-way or higher.

Q22. What is cache thrashing?

When the working set exceeds the cache size, continuously evicting and reloading the same data. Leads to near-zero cache hit rate. Fix: reduce working set size, partition data to fit in cache, use cache-oblivious algorithms.

Q23. What is a cache-oblivious algorithm?

An algorithm that performs well at all cache levels without knowing cache sizes — typically divide-and-conquer algorithms that recursively partition data into smaller sub-problems until they fit in each cache level (e.g., recursive matrix multiply, merge sort).

Q24. How does Java's volatile affect CPU caches?

volatile write inserts a StoreLoad barrier (full memory fence on x86). Ensures the write is flushed to RAM (visible to all threads). volatile read sees the most recent write from any thread. No CPU or compiler reordering around volatile accesses.

Q25. What is write buffering in CPUs?

CPUs buffer store operations in a store buffer to avoid blocking on cache coherency. Writes drain to L1 asynchronously. Can cause visibility issues between threads — requires memory barriers to flush the store buffer.

Q26. What is the impact of object layout on cache performance in Java?

JVM object layout: object header (12-16 bytes) + fields. Mark word (8 bytes) + class pointer (4 bytes compressed) = 12 bytes header. Field ordering is JVM-determined (not declaration order). Padding to 8-byte alignment wastes cache space for small objects.

Q27. What is -XX:+PrintFieldLayout JVM flag?

Prints the memory layout of Java objects (field offsets, padding). Useful for diagnosing false sharing and understanding object memory footprint. Requires `-XX:+UnlockDiagnosticVMOptions`.

Q28. What is the JOL (Java Object Layout) tool?

Java Object Layout library — programmatically inspects object layout: `ClassLayout.parseInstance(obj).toPrintable()` shows all field offsets, gaps, and padding.

Q29. How does garbage collection interact with CPU caches?

GC compaction improves spatial locality — objects allocated together stay together in the heap. However, GC pause flushes CPU caches (during stop-the-world). ZGC/Shenandoah minimize pauses to reduce cache disruption.

Q30. What is memory bandwidth and why does it matter?

The rate at which data can be transferred between RAM and CPU (typically 50-100 GB/s per socket). Memory-bound algorithms (sequential array scans) are limited by bandwidth. CPU-bound algorithms (compute on cached data) are not.

Q31. What is the Roofline model?

A performance model that classifies algorithms as memory-bound or compute-bound based on their arithmetic intensity (FLOPs per byte). Algorithms below the roofline are memory-bound; above are compute-bound. Guides optimisation strategy.